University of South-Eastern Norway

OAP200V

Object-oriented analysis, design and programming

09 – Introduction to JavaFX

Lecturer: Boban Vesin

JavaFX

- Java library for building modern, rich graphical user interfaces
- successor to Swing
- offers a more sophisticated set of features to create desktop applications with a more dynamic, web-like user experience

JavaFX vs Swing

Feature	JavaFX	Swing
Styling	CSS support	No CSS, styling programmatically
Layout Design	FXML and Scene Builder support	No FXML; UI defined in Java code
Graphics/Animation	Built-in animation and effects	Limited, requires custom coding
Multimedia	Built-in media support	No media support; requires libraries
Web Content	WebView for HTML/CSS content	Limited web capabilities (e.g., JEditorPane)
Concurrency	Task and Service for background processing	SwingWorker for background processing
Performance	GPU-accelerated rendering	CPU-based rendering
Development Status	Actively maintained (OpenJFX)	Legacy, no major updates

Key Components and Structure of JavaFX

- JavaFX applications are structured around several main components that allow for modular and powerful GUI design
- **Stage and Scene**
  - **Stage**: Represents the main window or a top-level container. It acts as the outer frame of your application.
  - **Scene**: A scene is like the content container of a stage, containing all UI elements (buttons, text fields, etc.). It’s assigned to the stage and holds the entire visual structure of your application.

Key concepts - Application

- Application is an abstract class in JavaFX that serves as the entry point for all JavaFX applications.
- Every JavaFX app extends this Application class and overrides its start method, which is called automatically when the application launches.
- The Application class manages the application lifecycle, including init(), start(), and stop() methods:
  - **init()**: Called before start(), typically used to initialize resources.
  - **start(Stage primaryStage)**: The main entry point for the JavaFX application where the primary stage (window) is configured.
  - **stop()**: Called when the application exits, allowing cleanup of resources.

Key concepts - Stage

- A Stage is a top-level JavaFX container or "window" where you can display content.
  - The primary stage is provided as a parameter to the start method by the JavaFX runtime.
- Properties of a Stage include:
  - **Title**: Set with stage.setTitle("My Application").
  - **Resizable**: Control whether the window can be resized.
  - **Show and Close**: Use stage.show() to display the window and stage.close() to close it.

Key concepts - Scene

- A Scene represents the content to be displayed inside a Stage.
- It contains a hierarchical scene graph, with a root node (often a layout container like VBox or Pane) that can hold other nodes, such as buttons, labels, and other controls.
- You can switch scenes on a stage or set new scenes dynamically during runtime.

Key Components and Structure of JavaFX

- **UI Controls**
  - JavaFX provides a rich set of **UI controls** (buttons, labels, text fields, tables, lists, etc.) that make it easy to build user interfaces. These controls are highly customizable and can be styled using CSS.
  - Common controls include Button, Label, TextField, TableView, and ListView, which allow users to interact with the application.

Layout Containers

- **Layouts** organize UI controls within a scene. JavaFX has several layout classes that help arrange components in different styles, such as:
  - HBox (horizontal box): Arranges elements in a single row.
  - VBox (vertical box): Arranges elements in a single column.
  - BorderPane: Divides the layout into five regions (top, bottom, left, right, center).
  - GridPane: Arranges elements in a grid of rows and columns.
  - StackPane: Stacks elements on top of each other.
- These layout managers allow to structure application's UI for easy and adaptive design.

Events and Event Handling

- JavaFX follows an **event-driven programming model** where user actions, like clicks or key presses, generate events.
- Each control (like Button or TextField) can have an event handler that defines how the application should respond to the user's actions.
- JavaFX uses Java 8’s lambda expressions for concise event handling, making code easier to read and write.

CSS Styling and FXML Markup

- **CSS Styling**: JavaFX components can be styled using CSS, allowing for separation between design and logic. This makes it easy to create visually appealing applications without hard-coding styles in Java.
- **FXML**: JavaFX supports **FXML**, an XML-based markup language that allows developers to define the UI structure separately from the application logic. This is similar to HTML for web development, where UI elements and layouts can be declared in a .fxml file and loaded into the Java application.

Animation and Effects

- JavaFX has a robust set of **animation classes** that enable animations like fading, scaling, rotating, and translating elements. This makes it easy to add dynamic effects, transitions, and visual feedback.
- **Effects** such as shadows, blurs, and color adjustments can be applied to controls, giving applications a polished, modern look.

Media and Graphics

- JavaFX includes powerful libraries for handling multimedia (audio and video) and custom 2D and 3D graphics. This allows for rich content within applications, from simple media playback to complex graphical rendering.
- Classes like MediaPlayer, Canvas, and 3D shapes (e.g., Box, Cylinder) enable multimedia and graphics capabilities.

WebView for Embedding Web Content

- JavaFX's WebView component allows embedding web pages or web-based content within the JavaFX application. This is useful for integrating web-based applications, HTML content, or online resources directly into your GUI.

Key Features and Advantages of JavaFX

- **Modern UI Elements and Animations**: JavaFX supports a wide range of controls and effects, making it suitable for creating visually appealing and interactive UIs.
- **Separation of Logic and UI**: With FXML and CSS, JavaFX allows clear separation between application logic (Java code) and UI (FXML/CSS), making the application easier to design and maintain.
- **Cross-Platform**: JavaFX applications run on Windows, macOS, and Linux, enabling cross-platform support with a consistent user experience.
- **Event-Driven Model**: JavaFX's event handling system is efficient, with lambda expressions providing a clean syntax.
- **Integration with Web Content**: JavaFX’s WebView allows embedding web content, which is useful for displaying dynamic content and integrating web applications.

Practical tutorial

Create a Maven project

- **Eclipse → File → New → Maven Project**
  - Check “Create a simple project (skip archetype selection)”
  - Group Id: com.example
  - Artifact Id: hellofx
  - Package: com.example.hellofx
    Finish.

REPLACE POM.XML WITH THIS

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
      https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>JavaFXExample</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>22</maven.compiler.source>
    <maven.compiler.target>22</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.openjfx</groupId>
      <artifactId>javafx-controls</artifactId>
      <version>22</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.openjfx</groupId>
        <artifactId>javafx-maven-plugin</artifactId>
        <version>0.0.8</version>

        <configuration>
          <mainClass>com.example.App</mainClass>
        </configuration>
      </plugin>
    </plugins>
  </build>

</project>
```

Create first window

- Create a package
- Create a class named App (for example)
- Add a following code
- Choose Run as maven build
  - In Goals enter javafx:run

```
package com.example;

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class App extends Application {

    @Override
    public void start(Stage stage) {

        Button button = new Button("Hello JavaFX!");

        StackPane root = new StackPane(button);

        Scene scene = new Scene(root, 400, 250);

        stage.setTitle("My First JavaFX Application");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```

Run it

- **From Eclipse using Maven:**
- **Run → Run Configurations… → Maven Build**
  - **Base directory**: your project
  - **Goals**: clean javafx:run
  - **Apply → Run`

Or: choose one of the tutorials

- https://www.youtube.com/watch?v=nz8P528uGjk
- https://www.youtube.com/watch?v=PbrKf7d566w
- https://www.youtube.com/watch?v=NjxwlB4nWI8

- Note: I recommend creating JavaFX project using Maven in Eclipse

Questions?